Cypher

Deep Packet Inspection & Hybrid Network Threat Intelligence

Resume Project Blueprint · C++ + Python FastAPI + Next.js / React · Antigravity-assisted

5 Phases	15+ Features	3 Languages	Hybrid Threat AI

PROJECT OVERVIEW

Cypher is a heavily customized fork of an open-source C++ Deep Packet Inspection engine. It adds a full-stack web dashboard (Next.js or React), a Python FastAPI layer, hybrid threat detection (rule-based + ML trained on your own browsing data), traffic visualization, and PCAP replay — transforming a CLI academic demo into a practical network analysis tool.

Base Project	Your Addition	Resume Signal
C++ CLI DPI engine, static .pcap files	Next.js/React dashboard + FastAPI bridge + hybrid threat engine	Full-stack, Systems, Security
Basic app classification (YouTube, Facebook)	Expanded signatures (50+ apps) trained from your own browser traffic	Protocol analysis, real data, DPI

No security features	Hybrid: rule-based fast-path + ML anomaly model (sklearn/isolation forest)	Cybersecurity, ML, anomaly detection
No interface	Dashboard: packet table, protocol charts, threat panel, export	Product thinking, UX, full-stack

ARCHITECTURE & TECH STACK

Layer	Technology	Role
Core Engine	C++17 (existing codebase)	Packet parsing, SNI extraction, flow tracking, multi-threaded DPI
API Layer	Python 3 + FastAPI + uvicorn	Runs C++ binary, streams JSON results via SSE, hosts threat detection logic
Frontend	Next.js (recommended) or React + Vite	Dashboard: packet table, charts, threat panel, export. See 'Frontend Choice' note below.
Threat Engine	Python: rules + scikit-learn	Rule-based fast-path + IsolationForest anomaly model trained on your own PCAP data
Test Data	generate_test_pcap.py + your browser capture	Synthetic scenarios + real SNI data you collect by browsing — this IS your training set

~~ FRONTEND CHOICE — Next.js vs React/Vite: Next.js is the stronger resume signal (SSR, file-based routing, App Router — interviewers recognise it). Use it if you're comfortable. If you want zero config and faster scaffolding today, go React + Vite. Both talk to FastAPI identically via fetch/SSE. The dashboard components are identical either way.

```
>> C++ → JSON stdout → FastAPI subprocess → SSE stream → Next.js / React dashboard
```

!! No live packet capture in this version. Cypher analyzes .pcap files with timestamp replay — giving 95% of the visual impact at 5% of the setup pain. Live capture (libpcap) is a clean Phase 2 you can mention on your resume as 'planned extension.'

DEVELOPMENT PHASES

PHASE 1 **Deep-Dive the Codebase [~45 min]**

1. Read & trace the packet lifecycle

- Clone the repo, read the README fully — it documents every file's purpose
- Open src/main_working.cpp first (simpler), then src/dpi_mt.cpp (multi-threaded)
- Trace one packet: PcapReader → PacketParser → SNIExtractor → FiveTuple → Flow → Block/Forward
- Understand include/types.h — AppType enum, FiveTuple struct, Flow struct
- Build and run it against the included test_dpi.pcap to see the terminal output

2. Understand the synthetic test data

- generate_test_pcap.py uses Scapy to craft fake packets — not real network traffic
- You will extend this AND supplement it with your own browsing capture (Phase 2)
- Skim the existing output format — you'll be adding --json mode to emit structured data

PHASE 2 Extend the C++ Engine + Capture Your Data [~1.5 hrs]

1. Add JSON output mode

- Add a --json flag to main_working.cpp (simpler version — modify this one)

```
{"ts":1.23,"src":"192.168.1.5","dst":"142.250.x.x","sport":54321,"dport":443,"proto":"TCP","app":"YouTube","sni":"www.youtube.com","blocked":false}
```

- Use nlohmann/json (drop json.hpp into include/) or simple manual string formatting
- Every packet emits one JSON line to stdout — the FastAPI layer reads this stream

2. Expand app signatures — as many as you want

- Add AppType entries for every app you browse: NETFLIX, ZOOM, DISCORD, TWITTER, CLOUDFLARE, AWS, GITHUB, WHATSAPP, TELEGRAM, SPOTIFY, TIKTOK, LINKEDIN, REDDIT, TWITCH, MICROSOFT, APPLE, FIGMA, NOTION, VERCEL, OPENAI, ANTHROPIC, STACKOVERFLOW, NPM, DOCKER, HEROKU, and any others you discover from your capture
- Pattern: sni.find('netflix') != npos → AppType::NETFLIX
- Also extend HTTP Host extraction for port-80 unencrypted traffic

The exact count doesn't matter — the story does. 'I captured my own browsing and added signatures for every service I actually use' is a strong interview answer

3. Capture your own browsing traffic — this is your ML training data

- Install Wireshark or use tcpdump (macOS/Linux) / npcap (Windows)

```
sudo tcpdump -i en0 -w my_traffic.pcap (run for 10-15 min while you browse normally)
```

- Browse YouTube, GitHub, Spotify, Discord, LinkedIn, Netflix — whatever you use daily
- This PCAP becomes both your extended test file AND the dataset to train your anomaly model
- Also extend generate_test_pcap.py with threat scenarios: port scan (50 unique ports from one IP), DNS flood (200+ queries/sec), suspicious ports (4444, 6666, 31337)

PHASE 3 Python FastAPI + Hybrid Threat Engine [~1.5 hrs]

1. Create server.py — the API bridge

- pip install fastapi uvicorn scikit-learn pandas numpy
- POST /analyze — accepts pcap filename, spawns C++ binary, streams JSON via SSE
- GET /stats — aggregate stats: packet count, protocol split, threat count, top IPs, top apps
- GET /threats — list of all detected threat events with severity
- GET /export/csv and GET /export/json — full packet log downloads
- Add CORS middleware so your frontend can talk to it

2. Rule-based threat detection — the fast path

- Runs in Python as packets stream in — no model needed, instant results:
- PORT SCAN: src IP hits 20+ unique dst ports within any 30-packet window → HIGH severity
- DNS FLOOD: src IP sends 50+ UDP/53 packets within 5s → MED severity
- SUSPICIOUS PORT: any packet to ports 4444, 6666, 23, 31337, 1337 → HIGH severity
- DOMINANT IP: single src IP > 30% of total traffic → LOW severity flag
- BLOCKED DOMAIN HIT: C++ engine marks blocked:true → escalate to threat event

Rule output format: {type, src_ip, dst_port, description, severity, timestamp}

3. ML anomaly detection — the intelligence layer

- Use scikit-learn IsolationForest — an unsupervised anomaly detector, no labelled data needed
- Features per packet: dst_port, packet_size, inter-arrival time, app_type (encoded), protocol
- Training: run your my_traffic.pcap through the C++ engine with --json, collect the output, train IsolationForest on it — this is your 'normal' baseline

```
from sklearn.ensemble import IsolationForest; model = IsolationForest(contamination=0.05); model.fit(features_df)
```

- At inference time: each new packet gets scored; outliers (score < threshold) are flagged as ML_ANOMALY
- Combine with rules: if BOTH rule and ML flag the same src IP → escalate to CRITICAL
- Save model as model.pkl with joblib — load on server startup

Interview answer ready: 'I captured my own browsing to build a normal traffic baseline, trained an IsolationForest on packet-level features, and combined it with deterministic rules so the system has both speed and intelligence. The ML layer catches unknown patterns rules would miss.'

PHASE 4 Frontend Dashboard [~2.5 hrs]

1. Scaffold — Next.js (recommended) or React/Vite

- Next.js: npx create-next-app@latest cypher --tailwind --app
- React/Vite: npm create vite@latest cypher -- --template react + npm install tailwindcss
- Either way: npm install recharts lucide-react axios
- Layout: sidebar nav + main content area with tab switching between panels
- Color scheme: dark navy (#0A1628) background, cyan (#00B4D8) accents — keep it consistent with the Cypher brand

2. Packet Analysis Table (core component)

- Columns: Timestamp | Src IP | Dst IP | Protocol | App | SNI/Host | Status | Threat
- Feed via EventSource (SSE) — rows append to top as the C++ engine processes the PCAP
- Color-code: green row = forwarded, red = blocked, amber = threat flagged, purple = ML anomaly
- Search bar: filter by protocol, app name, or IP substring
- Cap at 500 visible rows with a 'Show all' toggle

3. Statistics Panel

- Protocol pie chart: TCP / UDP split (Recharts PieChart)
- Top apps bar chart: YouTube 34%, HTTPS 22%... (Recharts BarChart)
- Packets/sec line chart: rolling window, updates per SSE event (Recharts LineChart)
- Summary cards row: Total Packets | Forwarded | Blocked | Rule Threats | ML Anomalies

4. Threat Monitor Panel

- Dedicated tab: 'Threats'
- Alert cards: severity badge (CRITICAL/HIGH/MED/LOW) + threat type + src IP + description + timestamp
- Tag each threat as RULE-BASED or ML-ANOMALY or COMBINED — shows the hybrid system at a glance
- COMBINED (both fired) = red pulsing border — visually most impactful thing in the demo
- Threat count badge on the nav tab updates in real time

5. Controls & Export

- Top bar: PCAP file selector + 'Analyze' button + replay speed (1x / 5x / 10x)
- Export CSV and Export JSON buttons — downloads full packet log
- Model info card: shows IsolationForest training stats (n_samples, contamination, anomaly rate)

PHASE 5 Polish, README & Resume Copy [~45 min]

1. Write the README.md

- Hero: project name + one-liner + screenshot GIF (record with Kap / ShareX / OBS)
- Architecture diagram — ASCII is fine, draw.io is better
- Three sections: Setup (C++ build + Python server + frontend), How It Works (3 paragraphs), Features list
- Add a 'Dataset' section explaining you captured your own traffic — this is a differentiator

2. Resume bullets — use these phrasings

- Built Cypher: a network packet analysis platform (C++ DPI engine + Python FastAPI + Next.js) that inspects TCP/UDP flows, extracts TLS SNI for 50+ applications, and streams results live via Server-Sent Events
- Implemented a hybrid threat detection system combining deterministic rule-based detection with an IsolationForest ML model trained on self-captured browser traffic, achieving dual-layer anomaly classification with CRITICAL escalation on combined signals
- Designed a multi-threaded packet processing pipeline using producer-consumer architecture with consistent 5-tuple hashing for lock-free per-flow state management

Coverage: C++, systems, networking, security, ML, full-stack, real-time data — from one project

FEATURE CHECKLIST

#	Feature	Description	Layer
1	■ **JSON output mode**	*Add --json flag to C++ engine; one structured JSON line per packet to stdout*	C++
2	■ **Expanded app signatures (50+)**	*Add every app you browse — collected from your own traffic capture. No fixed number target.*	C++
3	■ **Enriched synthetic test PCAP**	*Extend generate_test_pcap.py: port scan scenario, DNS flood, suspicious port traffic*	PY
4	■ **Self-captured browsing dataset**	*tcpdump/Wireshark session of your real traffic — doubles as training data and extended test PCAP*	PY
5	■ **FastAPI SSE streaming endpoint**	*POST /analyze streams JSON events; GET /stats, /threats, /export endpoints*	API
6	■ **Port scan detection (rule)**	*src IP hitting 20+ unique dst ports in 30-packet window → HIGH severity*	RULE
7	■ **DNS flood detection (rule)**	*50+ UDP/53 packets from same src in 5s window → MED severity*	RULE
8	■ **Suspicious port detection (rule)**	*Traffic to ports 4444 / 6666 / 31337 / 1337 / 23 → HIGH severity*	RULE
9	■ **Dominant IP detection (rule)**	*Single source > 30% of total traffic volume → LOW severity flag*	RULE
10	■ **IsolationForest anomaly model**	*Trained on your own traffic PCAP; scores each new packet; outliers flagged as ML_ANOMALY*	ML
11	■ **COMBINED escalation**	*Rule + ML both fire on same source → CRITICAL alert with pulsing UI indicator*	ML
12	■ **Packet analysis table**	*SSE-fed, color-coded, searchable, 500-row cap, RULE/ML/COMBINED badges per row*	UI

13	■ **Protocol & app charts**	*TCP/UDP pie + top-10 apps bar chart + packets/sec rolling line chart (Recharts)*	UI
14	■ **Threat monitor panel**	*Severity-tagged alert cards with RULE vs ML vs COMBINED labeling*	UI
15	■ **CSV & JSON export**	*Full packet log downloadable from dashboard toolbar*	UI
16	■ **Replay speed control**	*1x / 5x / 10x — controls throttle on Python stream reader*	API

C++ Core engine **PY** Python / data **API** FastAPI layer **RULE** Rule-based **ML** ML model **UI** Frontend

INTERVIEW PREP — QUESTIONS YOU MUST OWN

Q: How does SNI extraction work?

A: TLS Client Hello is the very first packet in an HTTPS connection. Before encryption starts, the client sends the destination hostname in plaintext as extension type 0x0000. We parse raw TCP payload byte-by-byte: verify byte 0 = 0x16 (handshake record), byte 5 = 0x01 (Client Hello), skip variable-length fields (session ID, cipher suites, compression), then iterate extensions until we find type 0x0000 and read the hostname string at the offset specified by the SNI length field.

Q: Why C++ for the core engine?

A: Packet processing is I/O and CPU intensive at scale. C++ gives direct memory access, zero-copy buffer handling via raw pointers, and fine-grained thread control with no GIL. Python would add 10-100x overhead per packet. This is also my first substantial C++ project — I wanted to understand memory management and multi-threading at a level that JS never forces you to confront.

Q: Explain the multi-threaded architecture.

A: Producer-consumer pattern. A reader thread feeds packets into thread-safe queues guarded by mutex + condition_variable. Load balancer threads distribute flows using consistent hashing on the 5-tuple — so all packets of the same TCP connection always reach the same Fast Path thread. This maintains per-flow state (like 'have we seen the Client Hello yet?') without cross-thread locks. FP threads do DPI, writer thread flushes output.

Q: How does the hybrid threat detection work?

A: Two layers. First: rule-based detection runs immediately as packets stream in — port scan (unique dst ports per src), DNS flood (packet rate), suspicious ports (4444, 31337...). These are deterministic and instant. Second: an IsolationForest model trained on my own browsing data scores each packet on features like dst_port, inter-arrival time, packet size, and app_type encoding. Outliers are flagged ML_ANOMALY. When both layers fire on the same source simultaneously, the event escalates to CRITICAL — that combined signal is much more reliable than either alone.

Q: Why IsolationForest specifically?

A: Unsupervised — I don't have labelled 'attack' data, just normal traffic. IsolationForest works by randomly isolating data points: anomalies are easier to isolate (shorter path in the tree), so they get high anomaly scores. It's O(n log n) to train, handles high-dimensional data well, and scikit-learn's implementation is production-stable. I trained it on my own browsing capture so the baseline reflects traffic patterns I'd actually see, not a generic dataset.

Q: What is a Five-Tuple?

A: Src IP, dst IP, src port, dst port, protocol — the 5 values that uniquely identify a network flow. It's the hash key in our flow table. Consistent hashing on the five-tuple guarantees all packets of one TCP session go to the same processing thread, preserving stateful information across packets without synchronisation overhead.

TODAY'S SHIPPING TIMELINE

Time	Block	Deliverable
0:00 – 0:45	Phase 1 — Codebase deep dive	Can explain every file's purpose without notes
0:45 – 2:15	Phase 2 — C++ + data capture	JSON mode working, signatures expanded, browsing PCAP captured
2:15 – 3:45	Phase 3 — FastAPI + threat engine	SSE endpoint live, 5 rules running, IsolationForest trained and serving
3:45 – 6:15	Phase 4 — Dashboard	Packet table + charts + threat panel + export all working
6:15 – 7:00	Phase 5 — Polish & push	README with GIF, resume bullets written, pushed to GitHub

!! Antigravity: use it for frontend scaffolding, boilerplate, and CSS. NOT for C++ architecture. NOT for the ML pipeline logic. If you can't walk through a function line by line, rewrite it yourself. 'Walk me through your threat detection code' is the first thing an interviewer will ask.